# Mathematical Foundations, Architecture Diagrams, and State Analysis of Modern LLM Training Methods — Part 3: Data Generation, Curation, Distillation, Compression, Agentic, Multimodal, Knowledge Injection, and Deployment

Chief Strategist J

June 12, 2026

## Contents

# Part I
# Synthetic Data Generation

## Self-Instruct

### Mathematical Foundation

Self-Instruct bootstraps an instruction dataset from a small seed pool $\mathcal{S}$ by using the LLM itself as a generator. At each iteration, $k$ seed instructions are sampled and the model generates a new instruction $\hat{x}$, its input, and output:

$$(\hat{x}, \hat{i}, \hat{y}) \sim P_\theta(\cdot \mid \text{few-shot}(\mathcal{S})) \tag{1}$$

Generated examples pass a quality filter (ROUGE similarity $< \delta$ to existing pool, length check):

$$\mathcal{S} \leftarrow \mathcal{S} \cup \{(\hat{x}, \hat{i}, \hat{y})\} \quad \text{iff} \quad \max_{s \in \mathcal{S}} \text{ROUGE}(\hat{x}, s) < \delta \tag{2}$$

The growing pool is used to fine-tune $\theta$, improving the quality of subsequent generations.

### Architecture Flow Diagram

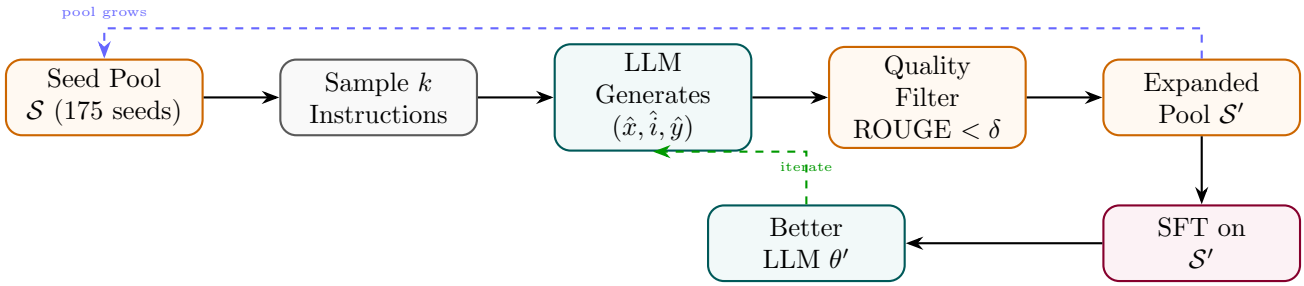


Figure 1: Self-Instruct: LLM generates new instructions from seed pool; quality filter expands the pool for SFT.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Pool size | 175 seed instructions | 52,000+ generated |
| Annotation source | Human-written seeds | LLM-generated |
| Diversity filter | N/A | ROUGE deduplication |
| Training data cost | Low (seeds only) | Essentially free to scale |
| Instruction following | Weak | Strong |

Table 1: State properties before and after Self-Instruct data generation.

# Evol-Instruct

## Mathematical Foundation

Evol-Instruct evolves existing instructions into harder variants via two mutation strategies. *In-depth* evolution adds constraints, increases steps, or requires deeper reasoning. *In-breadth* evolution creates new topic-adjacent instructions:

$$\hat{x} = M_{\text{depth}}(x) \quad \text{or} \quad \hat{x} = M_{\text{breadth}}(x), \qquad \hat{x} \sim P_{\text{teacher}}(\cdot \mid \text{evolve-prompt}, x) \tag{3}$$

Evolved instructions are validated (non-empty, no refusal, dissimilar from parent) and collected into $\mathcal{D}_{\text{evol}}$. Difficulty increases monotonically with evolution rounds $E$:

$$\mathbb{E}[\text{difficulty}(\hat{x}_E)] > \mathbb{E}[\text{difficulty}(\hat{x}_{E-1})] \tag{4}$$

## Architecture Flow Diagram

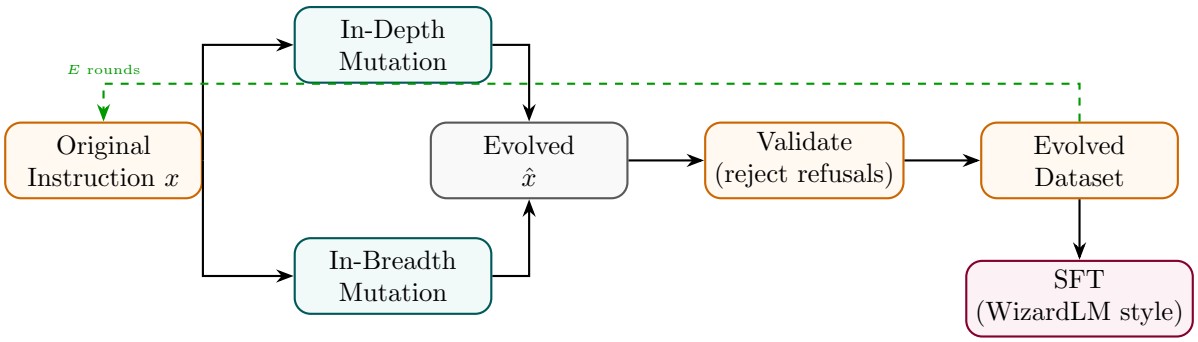


Figure 2: Evol-Instruct: in-depth and in-breadth mutations generate harder variants; validated pool used for SFT.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Instruction difficulty | Low (seed) | Progressively higher (per round $E$) |
| Instruction diversity | Limited | Broad via breadth mutations |
| Data source | Human-written | LLM-evolved |
| Evolution rounds | 0 | $E$ (typically 4–8) |
| Model performance | Baseline | WizardLM-style improvements |

Table 2: State properties before and after Evol-Instruct.

# Evol-Code

## Mathematical Foundation

Evol-Code applies the same evolutionary principle to programming tasks, using code-specific mutation operators: adding input/output constraints, combining two functions, increasing time/space complexity, or switching programming language:

$$\hat{p} = M_{\text{code}}(p), \quad M \in \{\text{constrain}, \text{combine}, \text{optimize}, \text{translate}\} \tag{5}$$

Each evolved problem $\hat{p}$ is solved by a teacher model, and the (problem, solution) pair is validated by unit-test execution:

$$(\hat{p}, \hat{s}) \in \mathcal{D}_{\text{code}} \iff \text{unit\_tests}(\hat{s}, \hat{p}) = \text{PASS} \tag{6}$$

## Architecture Flow Diagram

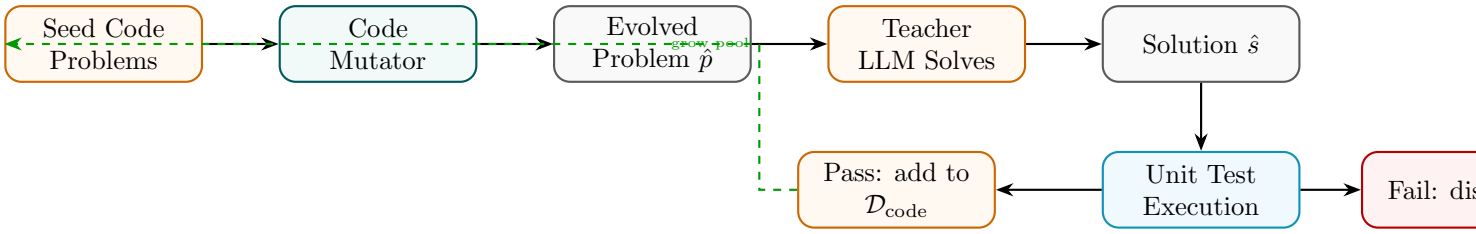


Figure 3: Evol-Code: code problems are mutated, solved by teacher, validated by unit tests before dataset inclusion.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Problem complexity | Simple seed tasks | Multi-constraint, optimised |
| Validation method | Human review | Automated unit tests |
| Data quality gate | None | Unit-test pass rate |
| Programming languages | Fixed | Multi-language via translate mutation |
| Code reasoning ability | Baseline | Significantly improved |

Table 3: State properties before and after Evol-Code data generation.

# Synthetic Data Generation

## Mathematical Foundation

A powerful teacher model $P_T$ generates synthetic (input, output) pairs from topic/schema prompts, which are then used to train a smaller student $P_S$. The synthetic data distribution approximates the real task distribution $P_{\text{real}}$:

$$\mathcal{D}_{\text{synth}} = \{(x_i, y_i) : x_i \sim P_{\text{topic}},\ y_i \sim P_T(\cdot \mid x_i)\} \tag{7}$$

Quality is improved by filtering on a scoring model or perplexity threshold $\tau_p$:

$$(x, y) \in \mathcal{D}_{\text{final}} \iff \text{score}(x, y) \geq \tau_s \ \text{ and } \ \text{PPL}(y) \leq \tau_p \tag{8}$$

## Architecture Flow Diagram

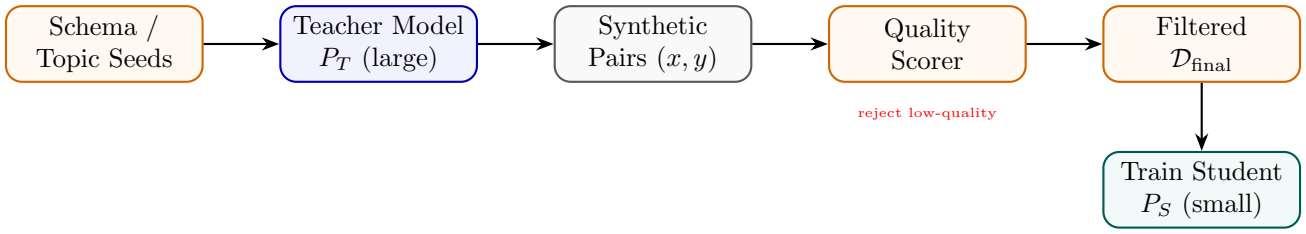


Figure 4: Synthetic Data Generation: teacher generates pairs from schema seeds; quality filter builds training set for student.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Data source | Human annotation | LLM-generated at scale |
| Annotation cost | High | Very low (API cost) |
| Data volume | Thousands | Millions scalable |
| Quality control | Human review | Automated scoring |
| Domain coverage | Limited | Any topic via schema |

Table 4: State properties before and after synthetic data generation.

# Rejection Sampling

## Mathematical Foundation

Rejection Sampling generates $N$ candidate completions from the current policy and keeps only those that pass a quality threshold, building a filtered fine-tuning corpus:

$$\mathcal{D}_{\text{RS}} = \{(x, y^{(k)}) : y^{(k)} \sim \pi_\theta(x),\ r(x, y^{(k)}) \geq \tau,\ k = 1..N\} \tag{9}$$

Expected acceptance rate at threshold $\tau$: $\alpha = P_{y\sim\pi_\theta}[r(x, y) \geq \tau]$. After SFT on $\mathcal{D}_{\text{RS}}$, the model's probability mass shifts toward high-reward completions:

$$\mathbb{E}_{y\sim\pi_{\theta'}}[r(x, y)] > \mathbb{E}_{y\sim\pi_\theta}[r(x, y)] \tag{10}$$

## Architecture Flow Diagram

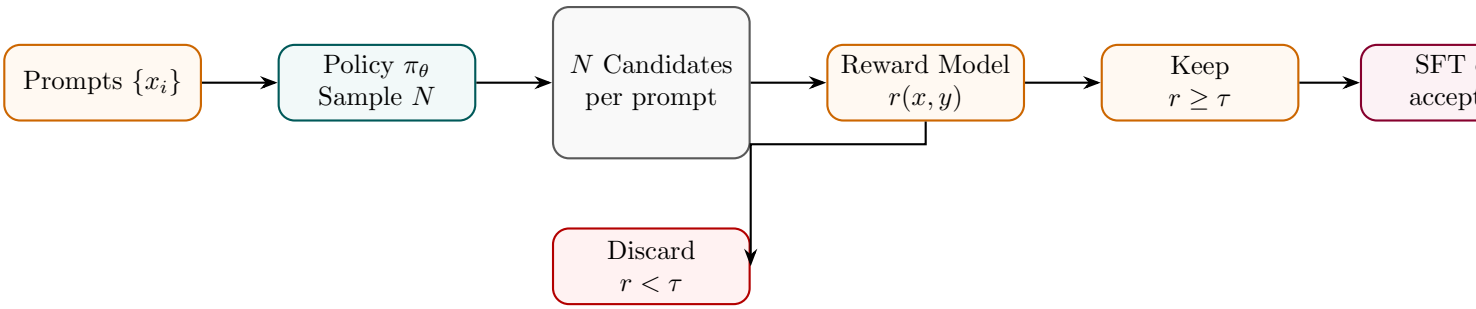


Figure 5: Rejection Sampling: $N$ completions generated; only those above reward threshold kept for SFT.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Training data quality | Mixed | High (above threshold $\tau$) |
| Samples per prompt | 1 | $N$ (with $\alpha N$ accepted) |
| Reward model | Optional | Required as filter |
| Iteration benefit | None | Compound with model updates |
| RL complexity | None | None (SFT only) |

Table 5: State properties before and after Rejection Sampling.

# Best-of-N Sampling

## Mathematical Foundation

Best-of-N (BoN) sampling selects the highest-reward completion from $N$ independent samples at inference time:

$$y^* = \arg\max_{k \in 1..N} r(x, y^{(k)}), \quad y^{(k)} \sim \pi_\theta(\cdot \mid x) \tag{11}$$

The expected reward improvement over single-sample baseline scales as:

$$\mathbb{E}[\max_k r(x, y^{(k)})] \approx \mu_r + \sigma_r \cdot \Phi^{-1}\left(\frac{N}{N+1}\right) \tag{12}$$

where $\mu_r, \sigma_r$ are the reward mean and std under $\pi_\theta$, and $\Phi^{-1}$ is the inverse normal CDF. BoN can also be used to build SFT datasets by collecting $(x, y^*)$ pairs.

## Architecture Flow Diagram

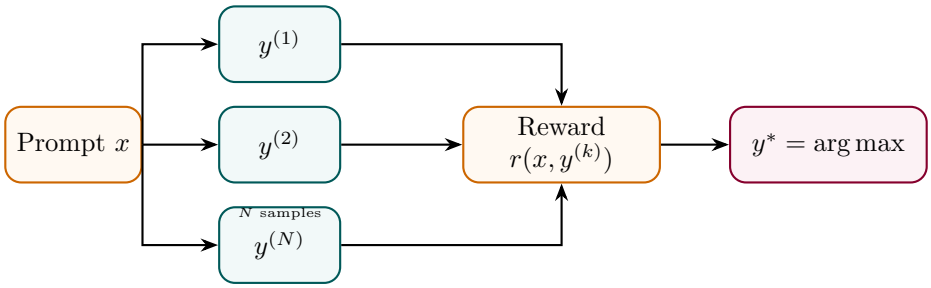


Figure 6: Best-of-N: $N$ samples scored by reward model; highest reward selected as final output.

## State Transition Table

| **Property** | **Before (single)** | **After (BoN)** |
|---|---|---|
| Samples per query | 1 | $N$ |
| Compute at inference | $1\times$ | $N\times$ |
| Expected reward | $\mu_r$ | $\mu_r + O(\sigma_r \log N)$ |
| Model weights | Unchanged | Unchanged |
| Best usage | Cheap inference | Quality-critical generation |

Table 6: State properties comparing single-sample and Best-of-N sampling.

# Part II
# Data Curation

## Data Filtering

### Mathematical Foundation

Data filtering removes low-quality examples from a raw corpus using a set of quality signals $\{q_1, \ldots, q_M\}$. A sample $(x, y)$ is accepted iff it passes all signal thresholds:

$$(x, y) \in \mathcal{D}_{\text{clean}} \iff \bigwedge_{m=1}^{M} q_m(x, y) \geq \tau_m \tag{13}$$

Common signals include perplexity under a small LM (detects gibberish), toxic content classifiers, length ratios, and language identification. A learned quality classifier $c_\phi$ can consolidate multiple signals:

$$c_\phi(x, y) = \sigma(w^\top [q_1(x, y), \ldots, q_M(x, y)]) \tag{14}$$

### Architecture Flow Diagram

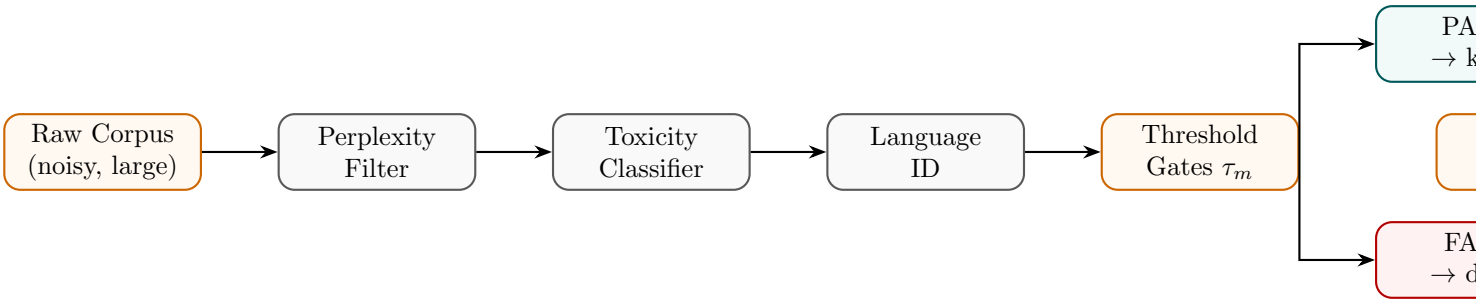


Figure 7: Data filtering pipeline: multiple quality signals applied in sequence; only passing samples retained.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Corpus size | Very large (noisy) | Smaller (high quality) |
| Noise level | High | Low |
| Domain coverage | Broad (noisy) | Focused (clean) |
| Filter signals | None | $M$ quality signals |
| Training efficiency | Low | Higher (less wasted compute) |

Table 7: State properties before and after data filtering.

# Deduplication

## Mathematical Foundation

Deduplication removes near-duplicate documents to prevent memorisation and improve sample diversity. MinHash LSH estimates Jaccard similarity between documents:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \approx \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}[h_k(A) = h_k(B)] \tag{15}$$

where $\{h_k\}$ are $K$ independent hash functions and $h_k(A) = \min_{w \in \text{ngrams}(A)} h_k(w)$. Documents with $J(A, B) > \tau_J$ are considered duplicates and one is removed:

$$\mathcal{D}_{\text{dedup}} = \{d_i : \forall j < i,\ J(d_i, d_j) \leq \tau_J\} \tag{16}$$

## Architecture Flow Diagram

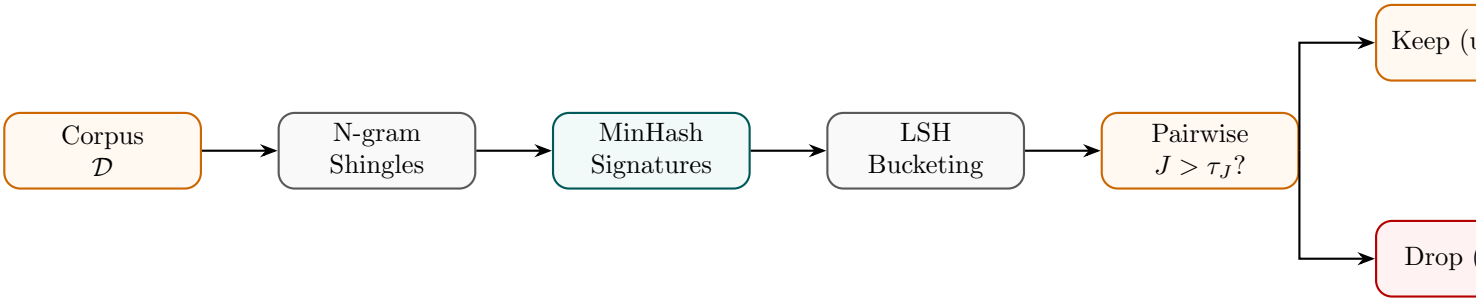


Figure 8: Deduplication: MinHash signatures group near-duplicates via LSH; high-similarity pairs discarded.

## State Transition Table

| **Property** | **Before** | **After** |
|---|---|---|
| Near-duplicate rate | 15–30% (web data) | Near zero |
| Memorisation risk | High | Reduced |
| Effective token diversity | Low | High |
| Corpus size | $\lvert\mathcal{D}\rvert$ | $\lvert\mathcal{D}_{\text{dedup}}\rvert < \lvert\mathcal{D}\rvert$ |
| Compute wasted on duplicates | High | Eliminated |

Table 8: State properties before and after deduplication.

# Active Learning

## Mathematical Foundation

Active Learning selects the most *informative* unlabelled examples for annotation, maximising model improvement per annotation dollar. Uncertainty sampling selects $x^*$ with highest predictive entropy:

$$x^* = \arg\max_{x \in \mathcal{U}} \mathbb{H}[P_\theta(y \mid x)] = \arg\max_{x} -\sum_{y} P_\theta(y|x) \log P_\theta(y|x) \tag{17}$$

Query-by-committee uses disagreement between an ensemble $\{\theta_1, \ldots, \theta_C\}$:

$$x^* = \arg\max_{x \in \mathcal{U}} \frac{1}{C} \sum_{c} \mathbb{KL}[P_{\theta_c}(\cdot|x) \| \bar{P}(\cdot|x)] \tag{18}$$

## Architecture Flow Diagram

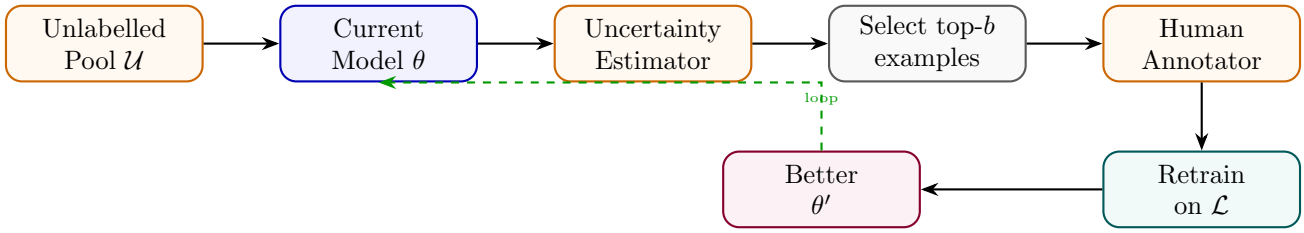


Figure 9: Active Learning: uncertainty estimator selects most informative examples; human annotates; model retrains.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Annotation strategy | Random sampling | Uncertainty-driven |
| Labels per accuracy gain | Many | Few (targeted) |
| Unlabelled pool | Fully unused | Iteratively queried |
| Annotation budget | Fixed | Allocated by informativeness |
| Model improvement per label | Low | High |

Table 9: State properties before and after Active Learning.

# Curriculum Learning

## Mathematical Foundation

Curriculum Learning trains on examples ordered by difficulty $d(x, y)$, starting with easy examples and progressively introducing harder ones. A competence function $c(t) \in [0, 1]$ controls what fraction of the difficulty range is accessible at step $t$:

$$c(t) = \min\left(1,\ c_0 + (1 - c_0)\frac{t}{T_{\text{ramp}}}\right) \tag{19}$$

The training distribution at step $t$ only samples from examples with $d(x, y) \leq c(t) \cdot d_{\max}$:

$$\mathcal{D}_t = \{(x, y) : d(x, y) \leq c(t) \cdot d_{\max}\}, \quad (x, y) \sim \text{Uniform}(\mathcal{D}_t) \tag{20}$$

## Architecture Flow Diagram

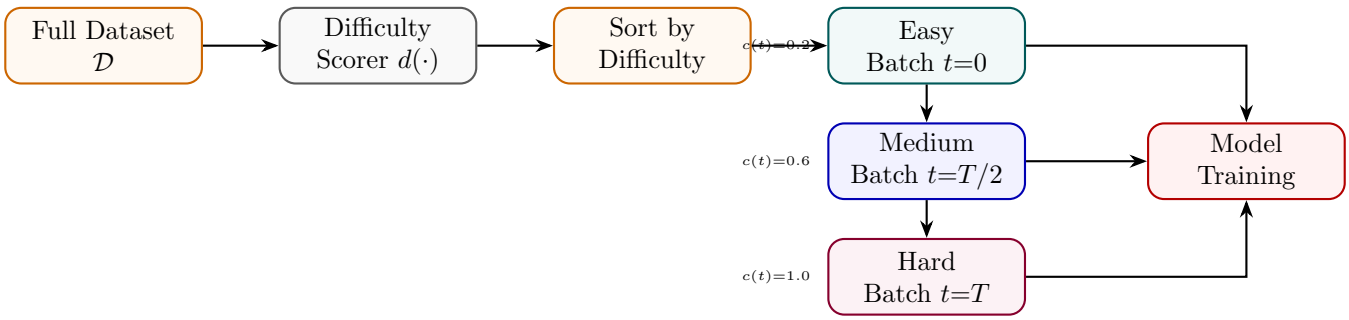


Figure 10: Curriculum Learning: examples sorted by difficulty; competence function progressively unlocks harder batches.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Training order | Random | Difficulty-ordered |
| Early training batches | Mixed difficulty | Easy only |
| Convergence speed | Slower | Faster (easier start) |
| Final performance | Baseline | Improved |
| Difficulty metric | N/A | Perplexity, length, or task-specific |

Table 10: State properties before and after Curriculum Learning.

# Part III
# Knowledge Distillation

## Knowledge Distillation

### Mathematical Foundation

Knowledge distillation transfers dark knowledge from a large teacher model $f_{\text{teacher}}$ to a compact student model $f_{\text{student}}$. Let $z_s$ and $z_t$ be the logits of the student and teacher models respectively. The soft probability distribution at temperature $T$ is computed as:

$$p_i(z, T) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{21}$$

The total distillation loss is a weighted combination of hard target cross-entropy and soft target Kullback-Leibler (KL) divergence:

$$\mathcal{L}_{\text{KD}} = (1-\alpha)\mathcal{L}_{\text{CE}}(y, p(z_s, 1)) + \alpha T^2 D_{\text{KL}}\big(p(z_t, T) \parallel p(z_s, T)\big) \tag{22}$$

where $\alpha \in [0, 1]$ balances the loss terms, and the temperature scaling factor $T^2$ scales the soft gradient updates to remain invariant of temperature changes:

$$\frac{\partial \mathcal{L}_{\text{KL}}}{\partial z_{s,i}} = \frac{1}{T}\big(p_i(z_s, T) - p_i(z_t, T)\big) \tag{23}$$

### Architecture Flow Diagram

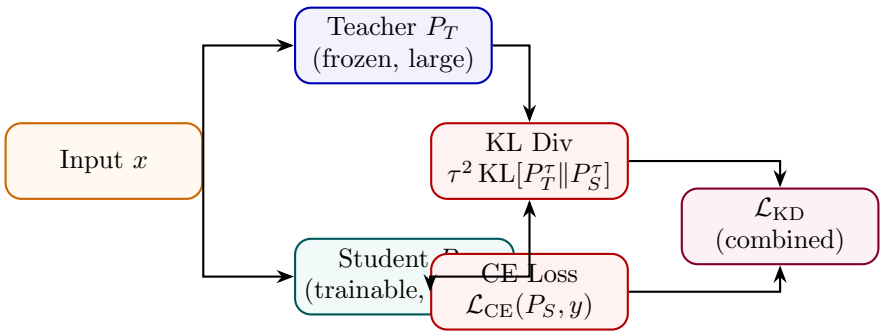


Figure 11: Knowledge Distillation: student learns from both hard labels (CE) and teacher soft logits (KL) simultaneously.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Student accuracy | Low (random) | Near-teacher accuracy |
| Training signal | Hard labels only | Hard labels + soft teacher logits |
| Temperature $\tau$ | N/A | $> 1$ (typically 2–5) |
| Dark knowledge transfer | None | Via inter-class similarities |
| Student size | Large (teacher) | Small ($\sim$10–100$\times$ fewer params) |

Table 11: State properties before and after Knowledge Distillation.

# Response Distillation

## Mathematical Foundation

Response Distillation trains the student by directly imitating the teacher's *full output sequence* rather than soft logits. The student minimises cross-entropy against teacher-generated responses:

$$\mathcal{L}_{\text{RD}} = -\sum_{(x,y_T)\in\mathcal{D}_T}\sum_{t}\log P_S(y_{T,t} \mid x, y_{T,<t}) \tag{24}$$

where $y_T \sim P_T(\cdot \mid x)$ are teacher completions. This is simpler than logit matching (no access to teacher logits required) and enables distillation from black-box APIs.

## Architecture Flow Diagram

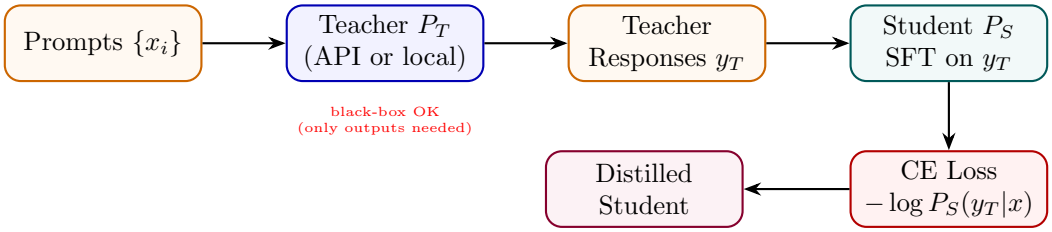


Figure 12: Response Distillation: teacher generates responses; student trained via SFT to imitate those outputs.

## State Transition Table

| **Property** | **Before (KD)** | **After (Response KD)** |
|---|---|---|
| Teacher access required | Logits (white-box) | Outputs only (black-box OK) |
| Loss type | KL on logits | CE on token sequence |
| Sequence-level knowledge | Partial | Full (complete generation) |
| Implementation complexity | High | Same as SFT |
| Applicable to proprietary APIs | No | Yes |

Table 12: State properties comparing logit-level KD and Response Distillation.

# Chain-of-Thought Distillation

## Mathematical Foundation

CoT Distillation transfers the teacher's *reasoning traces* to the student, so the student learns not just the answer but how to think step by step. The student is trained on $(x, c_T, y_T)$ triplets where $c_T$ is the teacher-generated chain:

$$\mathcal{L}_{\text{CoTD}} = -\sum_{(x,c_T,y_T)} \left[ \sum_m \log P_S(c_{T,m} \mid x, c_{T,<m}) + \sum_t \log P_S(y_{T,t} \mid x, c_T, y_{T,<t}) \right] \tag{25}$$

This allows a small student to match the accuracy of a large teacher that reasons step by step.

## Architecture Flow Diagram

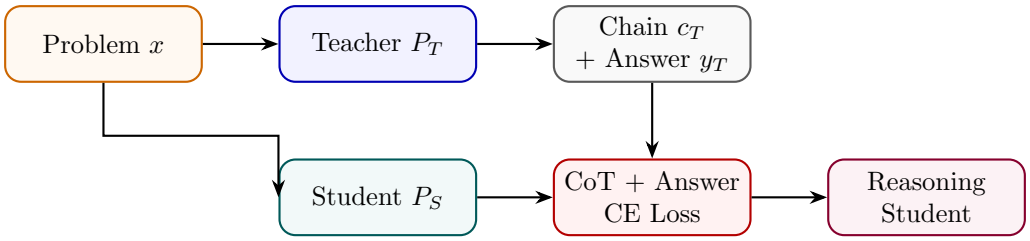


Figure 13: CoT Distillation: teacher's reasoning chains and answers jointly train student to produce similar traces.

## State Transition Table

| **Property** | **Before** | **After** |
|---|---|---|
| Student reasoning | Direct answer | Step-by-step chain |
| Training target | Answer only | Chain + answer |
| Reasoning data source | None | Teacher-generated |
| Multi-step problem accuracy | Low | Significantly higher |
| Output verbosity | Terse | Explanatory |

Table 13: State properties before and after CoT Distillation.

# Policy Distillation

## Mathematical Foundation

Policy Distillation minimises the KL divergence between a teacher policy $\pi_T$ and student policy $\pi_S$ over the full output distribution at every decoding step, not just the final token:

$$\mathcal{L}_{\text{PD}} = \mathbb{E}_{x\sim\mathcal{D}} \sum_t \mathbb{KL}[\pi_T(\cdot \mid x, y_{<t}) \| \pi_S(\cdot \mid x, y_{<t})] \tag{26}$$

Unlike response distillation (which uses sampled strings), policy distillation uses teacher token-level distributions and requires teacher logit access. This aligns the student's generation *process* with the teacher's, not just its outputs.

## Architecture Flow Diagram

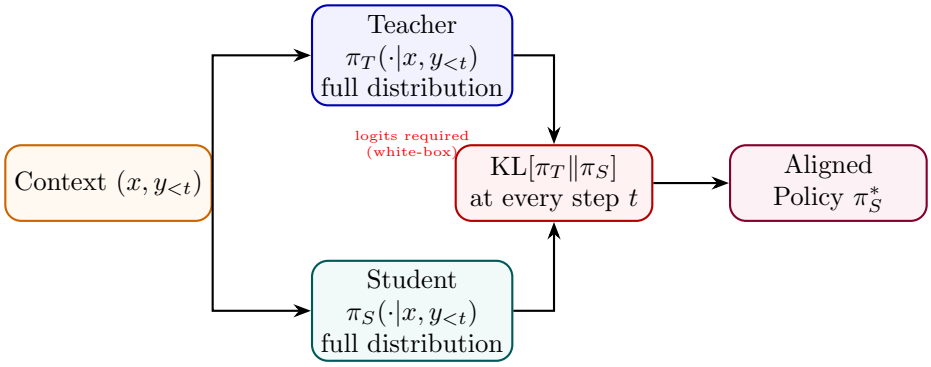


Figure 14: Policy Distillation: KL divergence between teacher and student distributions minimised at every token step.

## State Transition Table

| **Property** | **Before (Response KD)** | **After (Policy KD)** |
|---|---|---|
| Teacher signal | Sampled string | Full token distribution |
| KL divergence | N/A | Minimised per step |
| Teacher access | Black-box | White-box (logits) |
| Process alignment | Output only | Step-by-step |
| Overconfidence risk | Higher | Lower (distribution match) |

Table 14: State properties comparing Response and Policy Distillation.

# Reward Distillation

## Mathematical Foundation

Reward Distillation compresses a large teacher reward model $r_T$ into a smaller student reward model $r_S$ by regression on teacher-generated scores:

$$\mathcal{L}_{\text{RewardD}} = \mathbb{E}_{(x,y)}\left[(r_T(x,y) - r_S(x,y))^2\right] \tag{27}$$

The student can also be trained with a ranking-consistency loss to preserve relative orderings even if absolute score values differ:

$$\mathcal{L}_{\text{rank}} = -\log \sigma(r_S(x,y_w) - r_S(x,y_l)) \;\text{ whenever } r_T(x,y_w) > r_T(x,y_l) \tag{28}$$

## Architecture Flow Diagram

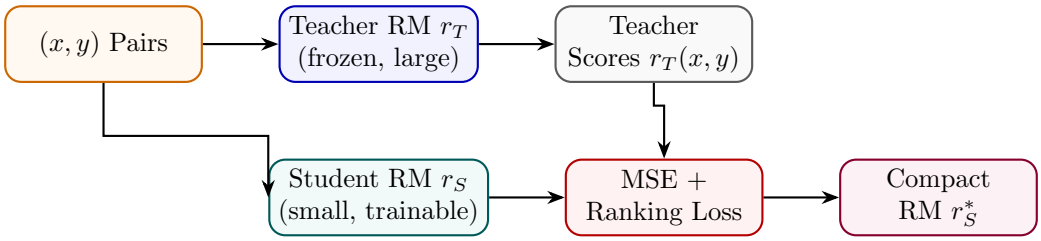


Figure 15: Reward Distillation: small student reward model trained via regression on large teacher reward scores.

## State Transition Table

| **Property** | **Before** | **After** |
|---|---|---|
| Reward model size | Large teacher | Small student |
| Inference latency | Slow | Fast |
| Score accuracy | High (teacher) | Near-teacher (distilled) |
| Training objective | Human preferences | Teacher score regression |
| Deployment cost | High | Low |

Table 15: State properties before and after Reward Distillation.

# Retrieval Distillation

## Mathematical Foundation

Retrieval Distillation trains a student retriever $q_S$ to match the relevance judgments of a larger teacher retriever $q_T$ (or cross-encoder) without requiring access to teacher gradients. The student minimises the KL divergence between teacher and student retrieval distributions over a document set $\mathcal{C}$:

$$\mathcal{L}_{\text{RD}} = \mathbb{KL}[P_T(\cdot \mid q) \| P_S(\cdot \mid q)], \quad P(d \mid q) \propto \exp(q_\phi(q)^\top e(d)) \tag{29}$$

where $q_\phi$ is the query encoder and $e(d)$ is the document embedding.

## Architecture Flow Diagram

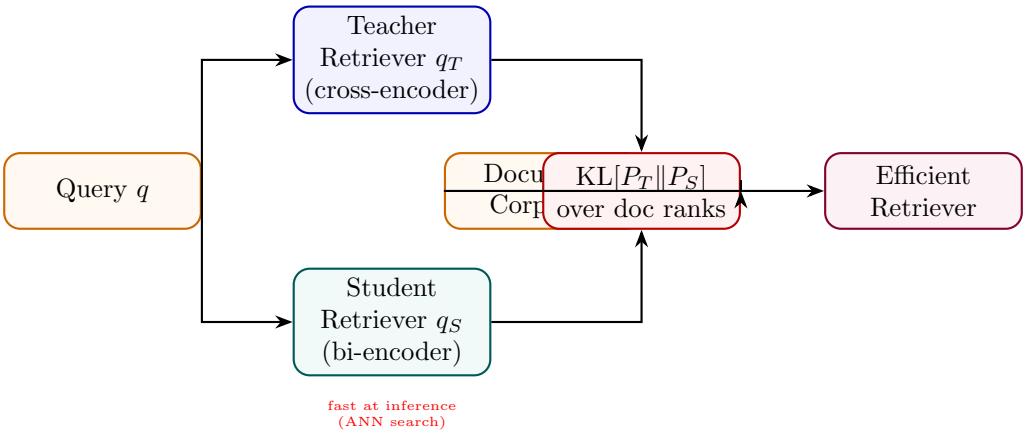


Figure 16: Retrieval Distillation: student bi-encoder trained to match teacher cross-encoder relevance distributions.

## State Transition Table

| Property | Before (Teacher) | After (Student) |
|---|---|---|
| Retriever type | Cross-encoder (slow) | Bi-encoder (fast ANN) |
| Query latency | $O(\lvert\mathcal{C}\rvert)$ | $O(\log \lvert\mathcal{C}\rvert)$ |
| Ranking quality | Very high | Near-teacher |
| Index requirement | None | Pre-computed embeddings |
| Training signal | Human judgments | Teacher soft scores |

Table 16: State properties comparing teacher cross-encoder and student bi-encoder after retrieval distillation.

# Part IV
# Model Compression

## Quantization-Aware Training (QAT)

### Mathematical Foundation

QAT simulates low-bit precision during model training, allowing the optimizer to adjust parameters to compensate for quantization errors. Let $w$ be the continuous weights. The quantized representation $\bar{w}$ is calculated as:

$$\bar{w} = S \cdot \text{clip}\left(\text{round}\left(\frac{w}{S}\right) - Z,\, q_{\min},\, q_{\max}\right) \tag{30}$$

where $S$ is the quantization scale factor, $Z$ is the zero-point, and $[q_{\min}, q_{\max}]$ defines the integer boundary (e.g. $[-128, 127]$ for 8-bit quantization). Because the rounding function has zero gradients almost everywhere ($\frac{\partial \text{round}(x)}{\partial x} = 0$), standard backpropagation fails. QAT resolves this by applying the Straight-Through Estimator (STE) during the backward pass, approximating the gradient of the rounding operator as identity:

$$\frac{\partial \bar{w}}{\partial w} \approx \begin{cases} 1 & \text{if } w \in [w_{\min}, w_{\max}] \\ 0 & \text{otherwise} \end{cases} \tag{31}$$

### Architecture Flow Diagram

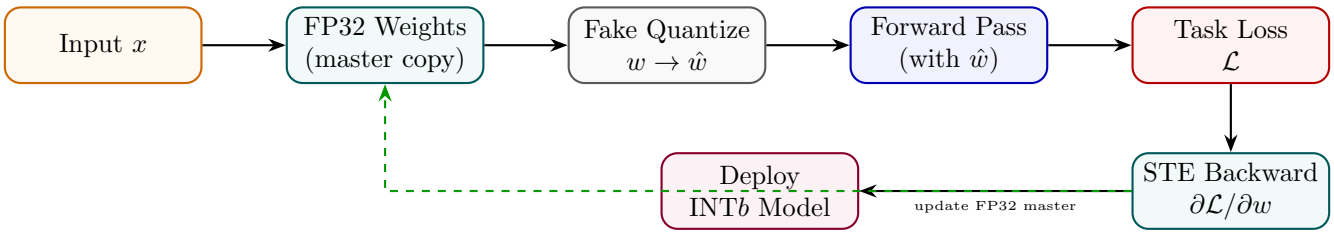


Figure 17: QAT: fake quantization in forward pass; STE allows gradients to flow to FP32 master weights.

### State Transition Table

| Property | Before (PTQ) | After (QAT) |
|---|---|---|
| Quantization awareness | None during training | Simulated in forward pass |
| Accuracy loss | 1–5% degradation | <1% typical |
| Calibration data | Small set required | Full training data |
| Weight format | FP32 | FP32 master, INT$b$ deploy |
| Gradient flow | Normal | Via STE |

Table 17: State properties comparing Post-Training Quantization and QAT.

# Pruning

## Mathematical Foundation

Pruning removes weights whose contribution to the model output is below a threshold, producing a sparse network. Magnitude-based unstructured pruning removes the fraction $p$ of weights with the smallest absolute values:

$$m_i = \mathbf{1}[|w_i| \geq \tau_p], \quad W_{\text{sparse}} = W \odot m \tag{32}$$

where $\tau_p$ is chosen to achieve sparsity ratio $s = 1 - \|m\|_1 / \|m\|_0$. Structured pruning removes entire heads or neurons (rows/columns) and is hardware-friendly:

$$\text{importance}(h) = \sum_{i \in h} |w_i| \cdot |\nabla_{w_i} \mathcal{L}| \tag{33}$$

## Architecture Flow Diagram

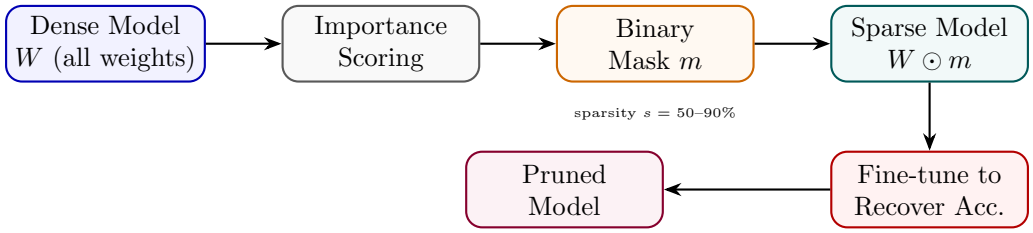


Figure 18: Pruning: importance scores determine binary mask; sparse model fine-tuned to recover accuracy.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Model sparsity | 0% | 50–90% |
| Parameter count | $\|\theta\|$ | $(1-s)\|\theta\|$ |
| Inference FLOPs | Baseline | Reduced (structured) |
| Accuracy | Baseline | Slightly reduced |
| Memory footprint | Full | Reduced by $s$ |

Table 18: State properties before and after pruning.

# Post-Training Quantization (PTQ)

## Mathematical Foundation

PTQ converts a trained FP32 model to reduced precision (INT8, INT4) without additional training. Per-channel scale factors $s_c$ are calibrated on a small unlabelled dataset $\mathcal{D}_{\text{cal}}$:

$$s_c = \frac{\max_{x \in \mathcal{D}_{\text{cal}}} |a_c(x)|}{2^{b-1} - 1}, \quad \hat{a}_c = \text{round}(a_c / s_c) \cdot s_c \tag{34}$$

GPTQ computes optimal quantization for each weight row by minimising the second-order reconstruction error using the Hessian:

$$\min_{\hat{W}} \|\hat{W}X - WX\|_F^2 \approx \sum_i (w_i - \hat{w}_i)^2 [H^{-1}]_{ii} \tag{35}$$

## Architecture Flow Diagram

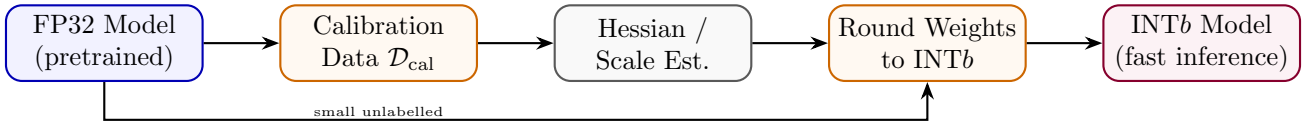


Figure 19: PTQ: calibration statistics guide per-channel scales; weights rounded to INT$b$ without retraining.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Weight precision | FP32 / BF16 | INT8 / INT4 |
| Memory usage | $4 \times n$ bytes | 1–0.5 $\times n$ bytes |
| Retraining required | N/A | None (calibration only) |
| Accuracy loss | 0% | 0.5–3% (INT8), higher INT4 |
| Calibration data | Not needed | Small sample (128–512 examples) |

Table 19: State properties before and after Post-Training Quantization.

# Part V
# Agentic Training

## Tool-Use Training

### Mathematical Foundation

Tool-Use Training teaches the model to generate structured tool invocations interleaved with reasoning (ReAct-style). The training data consists of trajectories $\tau = (t_1, a_1, o_1, \ldots, t_N, a_N, o_N, y)$ where $t_i$ is a thought, $a_i$ is a tool call, and $o_i$ is the observation. The model is trained to predict thought and action tokens conditioned on prior context and observations:

$$\mathcal{L}_{\text{tool}} = -\sum_{i=1}^{N} \sum_{j} \log P_\theta(t_{i,j} \mid \text{context}_{<i}) + \log P_\theta(a_{i,j} \mid \text{context}_{<i}, t_i) \tag{36}$$

### Architecture Flow Diagram

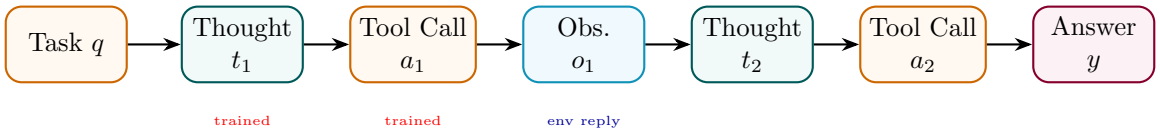


Figure 20: Tool-Use Training: thought-action-observation trajectory; model trained on thought and action tokens.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Output format | Free text | Thought + tool call interleaved |
| Tool invocation | None | Structured API calls |
| Observation integration | N/A | Conditioned on tool results |
| Multi-step planning | Weak | Strong |
| Training data type | Static QA | Trajectory (thought, action, obs) |

Table 20: State properties before and after Tool-Use Training.

# Function Calling Training

## Mathematical Foundation

Function Calling Training specifically trains the model to produce well-formed JSON function-call objects given a natural-language request and a tool schema. The model maps input $x$ and schema $S = \{f_1, \ldots, f_K\}$ to a structured call $c = \{\texttt{name}\text{: } f_k, \texttt{args}\text{: } \{a_1{:}v_1, \ldots\}\}$:

$$\mathcal{L}_{\text{FC}} = -\log P_\theta(c \mid x, S) \tag{37}$$

where $c$ is the ground-truth function call. Schema-aware token masking ensures the model only generates valid JSON that matches the function signature, reducing hallucination of non-existent arguments.

## Architecture Flow Diagram

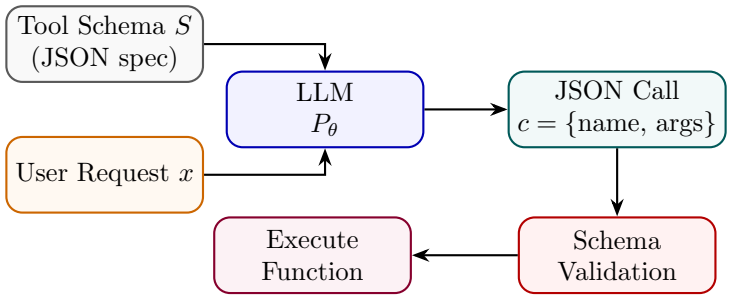


Figure 21: Function Calling Training: model receives request + schema, outputs structured JSON call validated against spec.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Output format | Natural language | Structured JSON |
| Schema adherence | None | Enforced |
| Argument hallucination | High | Low |
| API integration | Manual parsing | Direct execution |
| Training data | Text completions | (request, schema, call) triplets |

Table 21: State properties before and after Function Calling Training.

# Multi-Agent Training

## Mathematical Foundation

Multi-Agent Training trains a collection of specialised agents $\{\pi_1, \ldots, \pi_K\}$ and an orchestrator $\pi_0$ to collaboratively solve complex tasks. The orchestrator decomposes a task $T$ into sub-tasks $\{t_1, \ldots, t_K\}$ and assigns them to specialists:

$$\pi_0 : T \to \{(t_k, k)\}_{k=1}^{K}, \quad \pi_k : t_k \to y_k, \quad y = \pi_0(\{y_k\}) \tag{38}$$

Training uses task-completion reward at the full-task level, distributed to agents via credit assignment:

$$r_k = r_{\text{task}} \cdot \mathbf{1}[\pi_k \text{ contributed to solution}] \tag{39}$$

## Architecture Flow Diagram

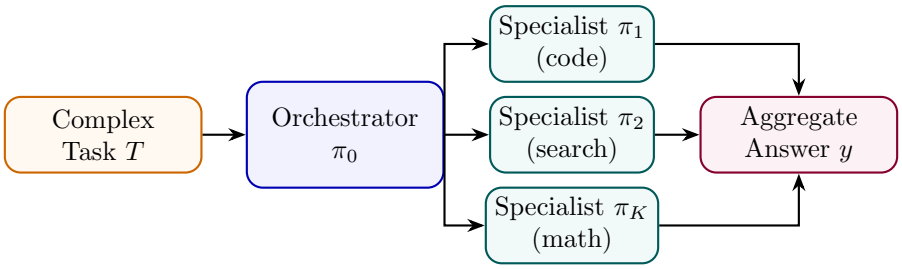


Figure 22: Multi-Agent Training: orchestrator decomposes task and assigns sub-tasks to specialised agents.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Task handling | Single monolithic model | Orchestrator + specialists |
| Specialisation | None | Per-domain agents |
| Credit assignment | N/A | Per-agent contribution |
| Task complexity ceiling | Single-context limit | Decomposable to multiple contexts |
| Coordination | N/A | Learned delegation protocol |

Table 22: State properties before and after Multi-Agent Training.

# Part VI
# Multimodal Training

## Vision-Language Training

### Mathematical Foundation

Vision-Language Training jointly trains a visual encoder $f_v$ and language model $f_l$ to process image-text pairs. A projection layer $P$ aligns visual features to the LLM's token embedding space:

$$v_{\text{tokens}} = P(f_v(I)), \quad \tilde{x} = [v_{\text{tokens}}; E(x_{\text{text}})] \tag{40}$$

Training uses a masked language modelling or auto-regressive loss on the text tokens conditioned on visual tokens:

$$\mathcal{L}_{\text{VL}} = -\sum_t \log P_{f_l}(x_t \mid v_{\text{tokens}}, x_{<t}) \tag{41}$$

### Architecture Flow Diagram

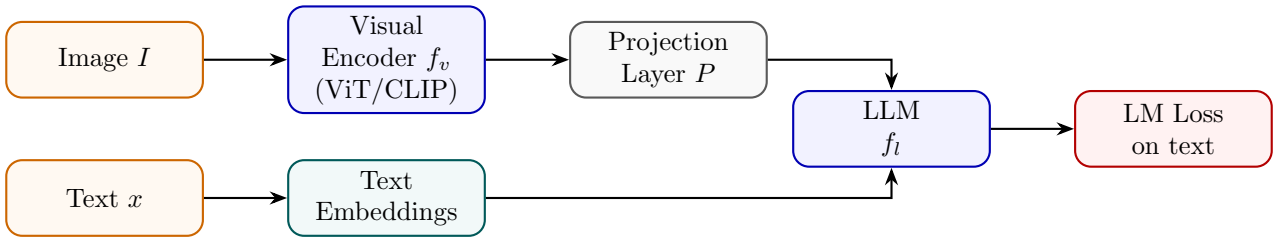


Figure 23: Vision-Language Training: visual encoder outputs projected to LLM token space; joint auto-regressive loss.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Input modality | Text only | Text + images |
| Visual understanding | None | Strong (via encoder) |
| Projection layer | Not present | Trained to align modalities |
| Alignment quality | N/A | Measured by VQA, captioning |
| Training data | Text corpora | Image-caption / VQA pairs |

Table 23: State properties before and after Vision-Language Training.

# Grounding Training

## Mathematical Foundation

Grounding Training teaches the model to link text spans to image regions (bounding boxes). Given a referring expression $e$ and image $I$, the model predicts box coordinates $(x_1, y_1, x_2, y_2)$ normalised to $[0, 1]^4$:

$$\hat{b} = f_\theta(I, e), \quad \mathcal{L}_{\text{GND}} = \text{L1}(\hat{b}, b^*) + \lambda(1 - \text{IoU}(\hat{b}, b^*)) \tag{42}$$

where $b^*$ is the ground-truth box and IoU is intersection-over-union. The model generates boxes as text tokens (serialised coordinates), enabling unified seq2seq training.

## Architecture Flow Diagram

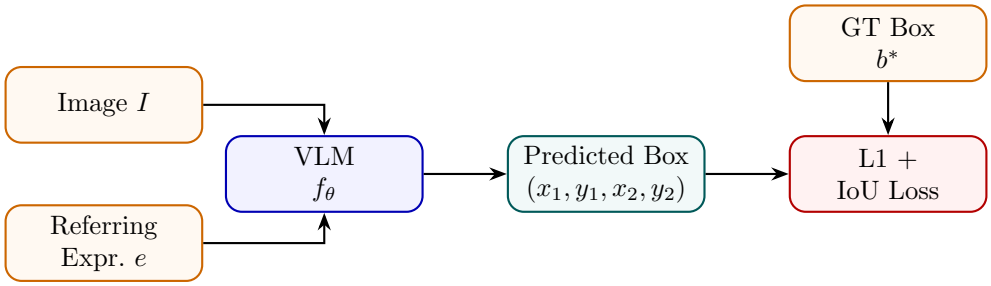


Figure 24: Grounding Training: VLM predicts bounding box coordinates from image and referring expression.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Output type | Text description | Bounding box coordinates |
| Spatial understanding | Implicit | Explicit (pixel-level) |
| Loss type | Cross-entropy | L1 + IoU |
| Grounding ability | None | Text-to-region linking |
| Training data | Captions | (image, expression, bbox) triplets |

Table 24: State properties before and after Grounding Training.

# Part VII
# Knowledge Injection

## Retrieval-Augmented Training

### Mathematical Foundation

Retrieval-Augmented Training (RAT) trains the model to use retrieved documents $\mathcal{R}(x) = \{d_1, \ldots, d_K\}$ as context. The model generates $y$ conditioned on the query and retrieved set:

$$P_\theta(y \mid x, \mathcal{R}(x)) = \prod_t P_\theta(y_t \mid x, d_1, \ldots, d_K, y_{<t}) \tag{43}$$

Training includes both retrieval-present and retrieval-absent examples to prevent over-reliance on context. A closed-book loss component ensures the model retains parametric knowledge:

$$\mathcal{L}_{\text{RAT}} = \lambda \, \mathcal{L}_{\text{open-book}} + (1 - \lambda) \, \mathcal{L}_{\text{closed-book}} \tag{44}$$

### Architecture Flow Diagram

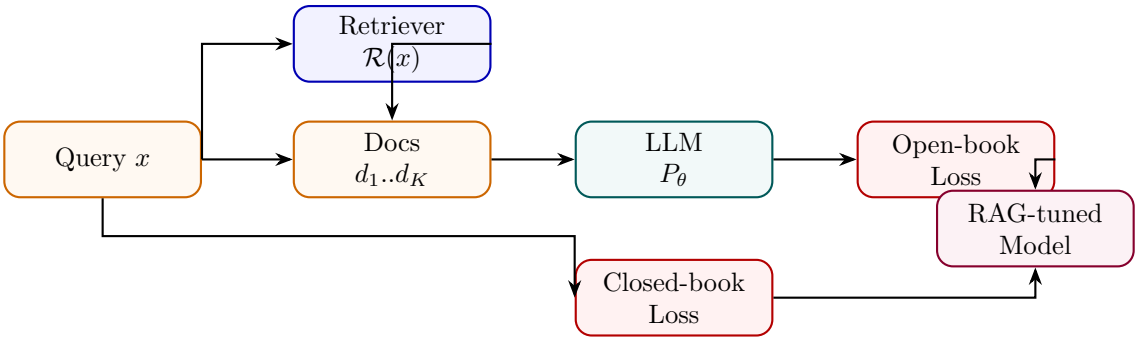


Figure 25: RAT: model trained on both retrieval-augmented and closed-book losses to balance open/parametric knowledge.

### State Transition Table

| Property | Before | After |
|---|---|---|
| Context awareness | Parametric only | Retrieved + parametric |
| Knowledge staleness | Fixed (training cutoff) | Dynamic (fresh retrieval) |
| Hallucination rate | Higher | Lower (grounded in docs) |
| Retriever dependency | None | Required at inference |
| Parametric retention | Full | Preserved (closed-book loss) |

Table 25: State properties before and after Retrieval-Augmented Training.

# Memory Tuning

## Mathematical Foundation

Memory Tuning trains the model to store and retrieve episodic memories $\mathcal{M} = \{(k_i, v_i)\}$ in an external key-value store. The model learns to write memories via a write head $w_\theta$ and retrieve via dot-product attention:

$$\text{retrieve}(q) = \sum_i \text{softmax}(q^\top k_i / \sqrt{d}) \cdot v_i, \quad q = f_\theta^{\text{query}}(x) \tag{45}$$

Training uses a distractor task: memory-relevant queries should prefer correct memory entries, trained with a contrastive retrieval loss:

$$\mathcal{L}_{\text{mem}} = -\log \frac{\exp(q^\top k^+ / \tau)}{\sum_j \exp(q^\top k_j / \tau)} \tag{46}$$

## Architecture Flow Diagram

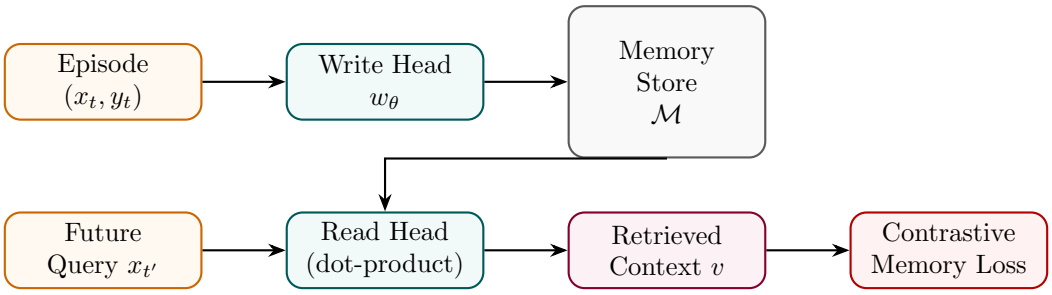


Figure 26: Memory Tuning: write head stores episodes; read head retrieves via attention; contrastive loss trains both.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Memory type | Parametric only | Parametric + episodic KV store |
| Context window | Fixed $T$ tokens | Unbounded (external memory) |
| Personalisation | None | Per-user episodic recall |
| Read/write mechanism | None | Learned attention-based |
| Long-term recall | Poor | Strong |

Table 26: State properties before and after Memory Tuning.

# Domain Knowledge Injection

## Mathematical Foundation

Domain Knowledge Injection embeds structured domain knowledge (ontologies, knowledge graphs, fact databases) into the model. Entities $e$ and relations $r$ from a KG are embedded alongside token embeddings and injected via cross-attention:

$$h'_t = h_t + \text{CrossAttn}(h_t, \{e_i, r_{ij}\}_{(i,j)\in\mathcal{G}}) \tag{47}$$

Training uses a knowledge grounding loss that verifies generated text against retrieved facts:

$$\mathcal{L}_{\text{KI}} = \mathcal{L}_{\text{LM}} + \mu\,\mathcal{L}_{\text{fact-check}} \tag{48}$$

## Architecture Flow Diagram

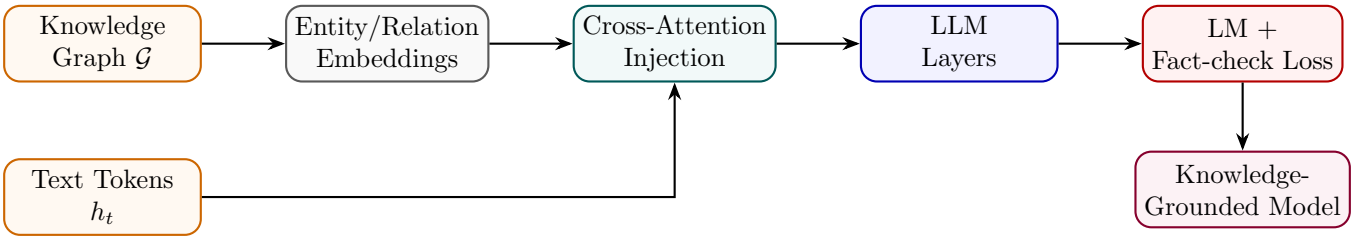


Figure 27: Domain Knowledge Injection: KG entity/relation embeddings injected into LLM via cross-attention.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Knowledge source | Implicit (pretraining) | Explicit KG entities |
| Fact accuracy | Moderate | Significantly higher |
| Knowledge update | Retrain required | KG update sufficient |
| Cross-attention overhead | None | One extra module per layer |
| Domain coverage | General | Domain-specific + general |

Table 27: State properties before and after Domain Knowledge Injection.

# Part VIII
# Deployment and Training Efficiency

## Mixed Precision Training

### Mathematical Foundation

Mixed Precision Training maintains FP32 master weights but computes forward and backward passes in FP16/BF16. A loss scaling factor $S$ prevents underflow of small gradients:

$$\tilde{\mathcal{L}} = S \cdot \mathcal{L}, \quad g_{\text{FP32}} = \frac{1}{S} g_{\text{FP16}} \tag{49}$$

BF16 has the same exponent range as FP32 (8 bits) but fewer mantissa bits (7 vs 23), making it more numerically stable without loss scaling:

$$\text{memory(FP16)} = 2N \text{ bytes vs memory(FP32)} = 4N \text{ bytes} \tag{50}$$

### Architecture Flow Diagram

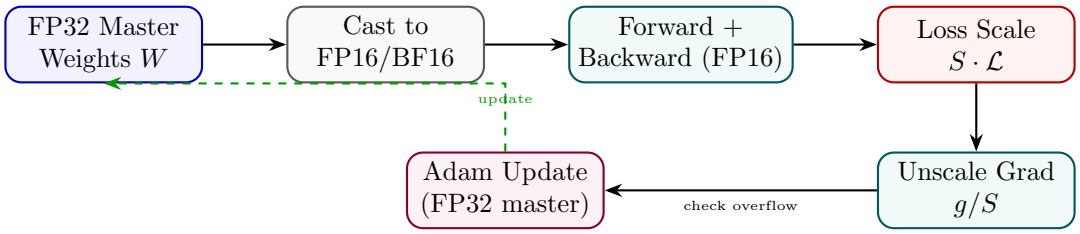


Figure 28: Mixed Precision Training: FP16 compute with FP32 master weights; loss scaling prevents gradient underflow.

### State Transition Table

| Property | Before (FP32) | After (Mixed FP16) |
|---|---|---|
| Compute precision | FP32 | FP16/BF16 |
| Memory footprint | $4N$ bytes | $2N + 4N$ (half + master) |
| Training speed | Baseline | 2–3× faster |
| Numerical stability | High | Managed via loss scaling |
| GPU tensor cores | Unused | Utilised (FP16/BF16 native) |

Table 28: State properties before and after Mixed Precision Training.

# Gradient Checkpointing

## Mathematical Foundation

Gradient Checkpointing trades compute for memory by not storing all intermediate activations during the forward pass. Instead, activations at checkpoint nodes $\mathcal{C} \subset \{1, \ldots, L\}$ are stored; all others are recomputed during the backward pass:

$$\text{Memory}_{\text{GC}} = O(\sqrt{L} \cdot d), \quad \text{vs} \;\; O(L \cdot d) \text{ without GC} \tag{51}$$

For $L$ layers, optimal checkpointing every $\sqrt{L}$ layers minimises memory while adding one extra forward pass overhead:

$$\text{Compute}_{\text{GC}} \approx 2 \cdot \text{Compute}_{\text{standard}} \tag{52}$$

## Architecture Flow Diagram

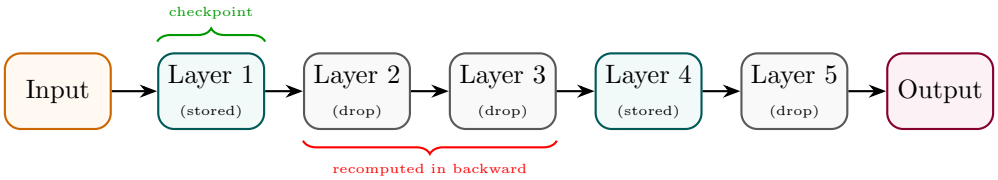


Figure 29: Gradient Checkpointing: only checkpoint activations stored; intermediate activations recomputed in backward.

## State Transition Table

| Property | Before | After |
|---|---|---|
| Activation memory | $O(L \cdot d)$ | $O(\sqrt{L} \cdot d)$ |
| Recomputation | None | Extra forward for non-checkpoints |
| Max batch size | Limited by activations | Larger (mem freed) |
| Compute overhead | Baseline | $\sim$30% more FLOPs |
| Large model feasibility | Memory OOM | Enabled |

Table 29: State properties before and after Gradient Checkpointing.

# Flash Attention

## Mathematical Foundation

FlashAttention accelerates attention computation by tiling input blocks in SRAM and utilizing online softmax reduction to avoid materializing the massive $N \times N$ attention matrix in HBM. Let $Q, K, V$ be split into blocks $Q_i, K_j, V_j$ of sizes $B_c, B_r$. The online softmax computes running scaling statistics. For a query block $Q_i$: At step $j$ (processing key block $K_j$), we compute raw attention scores:

$$S^{(j)} = \frac{Q_i K_j^\top}{\sqrt{d}} \tag{53}$$

We update running row-wise maximums $m^{(j)}$ and denominators $d^{(j)}$:

$$\tilde{m}^{(j)} = \max(S^{(j)}) \tag{54}$$

$$m^{(j)} = \max(m^{(j-1)}, \tilde{m}^{(j)}) \tag{55}$$

$$\tilde{d}^{(j)} = \sum_k \exp(S_k^{(j)} - m^{(j)}) \tag{56}$$

$$d^{(j)} = d^{(j-1)} \exp(m^{(j-1)} - m^{(j)}) + \tilde{d}^{(j)} \tag{57}$$

The running attention output block $O_i$ is updated dynamically using:

$$O_i^{(j)} = \frac{d^{(j-1)} \exp(m^{(j-1)} - m^{(j)}) \cdot O_i^{(j-1)} + \exp(\tilde{m}^{(j)} - m^{(j)}) \cdot \tilde{P}^{(j)} V_j}{d^{(j)}} \tag{58}$$

where $\tilde{P}^{(j)} = \exp(S^{(j)} - \tilde{m}^{(j)})$. This achieves exact attention outputs with $O(N)$ memory access instead of $O(N^2)$.

## Architecture Flow Diagram

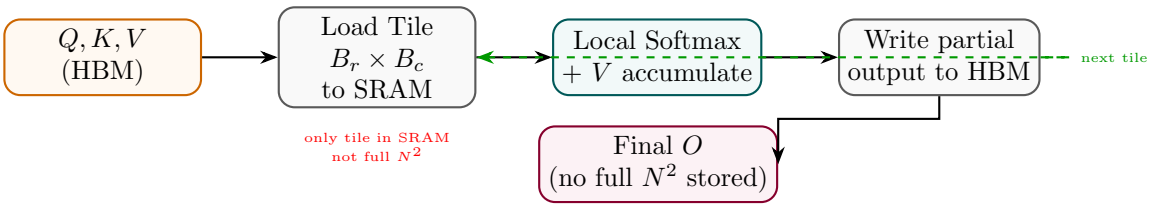


Figure 30: Flash Attention: tiled block-wise computation avoids materialising the full attention matrix in HBM.

## State Transition Table

| Property | Before (Standard) | After (Flash Attention) |
|---|---|---|
| Attention matrix memory | $O(N^2)$ HBM | $O(N)$ (tiles in SRAM) |
| HBM read/write passes | Multiple | Minimal (one pass) |
| Compute FLOPs | $O(N^2 d)$ | $O(N^2 d)$ (same) |
| Speed (wall-clock) | Baseline | 2–4× faster |
| Long-sequence feasibility | OOM at $N > 4096$ | $N > 100{,}000$ feasible |

Table 30: State properties comparing Standard and Flash Attention.

# Fully Sharded Data Parallel (FSDP)

## Mathematical Foundation

FSDP shards all three components of the model state (Parameters $\Psi$, Gradients $G$, and Optimizer States $O$) across all $N$ available GPUs. Let the model parameters occupy $M_{\text{param}} = 2\Psi$ bytes in 16-bit precision, gradients occupy $M_{\text{grad}} = 2\Psi$ bytes, and Adam optimizer states occupy $M_{\text{opt}} = 12\Psi$ bytes (storing FP32 copies of weights, first moments, and second moments). Under standard Distributed Data Parallel (DDP), the parameters, gradients, and optimizer states are fully duplicated on each GPU:

$$M_{\text{baseline}} = 2\Psi + 2\Psi + 12\Psi = 16\Psi \text{ bytes} \tag{59}$$

FSDP shards these states across $N$ GPUs. The peak memory occupancy per GPU is:

$$M_{\text{FSDP}} = \underbrace{\frac{2\Psi}{N}}_{\text{sharded params}} + \underbrace{\frac{2\Psi}{N}}_{\text{sharded grads}} + \underbrace{\frac{12\Psi}{N}}_{\text{sharded optimizer}} + \underbrace{2\Psi_{\text{block}}}_{\text{reconstructed block}} \tag{60}$$

During the forward pass, an 'All-Gather' operation reconstructs parameters block-by-block on the fly, which are discarded immediately after computing the block activation. During the backward pass, parameters are gathered again, gradients are computed, and a 'Reduce-Scatter' operation synchronizes and shards the gradients across GPUs.

## Architecture Flow Diagram

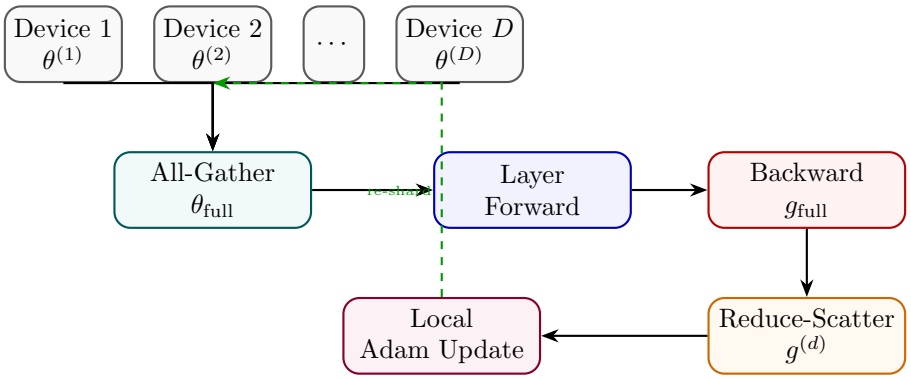


Figure 31: FSDP: parameters sharded across devices; All-Gather for compute, Reduce-Scatter for gradient aggregation.

## State Transition Table

| **Property** | **Before (DDP)** | **After (FSDP)** |
|---|---|---|
| Per-device param memory | $O(\|\theta\|)$ | $O(\|\theta\|/D)$ |
| Optimizer state memory | $O(\|\theta\|)$ per device | $O(\|\theta\|/D)$ per device |
| Communication pattern | AllReduce (gradients) | All-Gather + Reduce-Scatter |
| Max model size | GPU VRAM limited | $D\times$ larger |
| Training throughput | Baseline | Near-linear scaling |

Table 31: State properties comparing DDP and FSDP.

# DeepSpeed ZeRO

## Mathematical Foundation

ZeRO (Zero Redundancy Optimizer) partitions the memory footprint of the model across data-parallel processes. It consists of three successive stages:

1. **ZeRO-1 (Optimizer State Partitioning)**: The optimizer states (e.g. Adam states) are sharded across $N_{\text{dp}}$ data-parallel ranks. Parameters and gradients remain fully replicated. Peak memory per GPU is:

$$M_{\text{ZeRO-1}} = 2\Psi + 2\Psi + \frac{12\Psi}{N_{\text{dp}}} \tag{61}$$

2. **ZeRO-2 (Gradient Partitioning)**: Gradients are sharded immediately after their calculation in the backward pass. Peak memory per GPU is:

$$M_{\text{ZeRO-2}} = 2\Psi + \frac{2\Psi}{N_{\text{dp}}} + \frac{12\Psi}{N_{\text{dp}}} \tag{62}$$

3. **ZeRO-3 (Parameter Partitioning)**: Parameters are sharded across the ranks. The forward and backward passes reconstruct individual layer weights on the fly via 'All-Gather'. Peak memory per GPU is:

$$M_{\text{ZeRO-3}} = \frac{2\Psi}{N_{\text{dp}}} + \frac{2\Psi}{N_{\text{dp}}} + \frac{12\Psi}{N_{\text{dp}}} \tag{63}$$

Additionally, **ZeRO-Offload** offloads partitioned states to host CPU memory, utilizing PCIe bandwidth to scale training to trillion-parameter models on limited GPU nodes.

## Architecture Flow Diagram

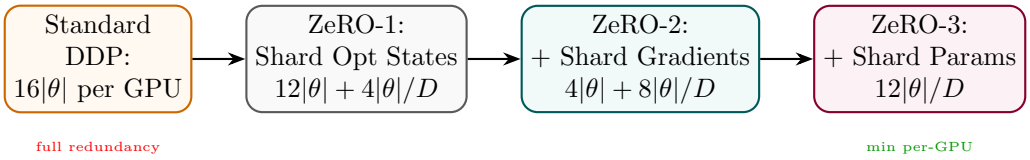


Figure 32: DeepSpeed ZeRO stages: each stage shards an additional component, progressively reducing per-GPU memory.

## State Transition Table

| **Property** | **Standard DDP** | **ZeRO-3** |
|---|---|---|
| Optimizer state memory | $8\|\theta\|$ per device | $8\|\theta\|/D$ |
| Gradient memory | $4\|\theta\|$ per device | $4\|\theta\|/D$ |
| Parameter memory | $4\|\theta\|$ per device | $4\|\theta\|/D$ |
| Total per device | $16\|\theta\|$ | $16\|\theta\|/D$ |
| Max trainable size | GPU VRAM bound | $D\times$ larger |

Table 32: Memory comparison between standard DDP and DeepSpeed ZeRO-3.